Package 'adabay'

August 4, 2026

Type Package

Title Rapid Bayesian Group Sequential Design Evaluation and Calibration

Version 0.1.0

Date 2026-08-04

Description Implements a semi-simulation framework for the rapid evaluation and calibration of Bayesian group sequential designs (GSDs). Virtual trials are simulated by Monte Carlo, and per-look posteriors are computed in closed form by approximating any user-specified prior with a finite mixture of conjugate components. Supports continuous (normal-normal or normal-inverse-gamma updates), binary (beta-binomial), count (gamma-Poisson) and time-to-event (gamma-exponential) primary endpoints, posterior-probability decision rules with efficacy and binding or non-binding futility stopping, and a precomputation strategy that decouples threshold and look-time selection from the simulation pass. Methods are described in He, Yu, Cro and Billot (2026) <doi:10.48550/arXiv.2607.22900>, and the package in He and Yu (2026) <doi:10.48550/arXiv.2608.01068>.

License MIT + file LICENSE

URL https://github.com/trilixlab/adabay, https://trilixlab.github.io/adabay/

BugReports https://github.com/trilixlab/adabay/issues

Depends R (>= 4.0.0)

Imports graphics, parallel, Rcpp (>= 1.0.7), stats

LinkingTo Rcpp

Suggests RBesT (>= 1.6), ggplot2, ggsci, testthat (>= 3.0.0), knitr, rmarkdown

Encoding UTF-8

Language en-GB

VignetteBuilder knitr

Config/testthat/edition 3

Config/roxygen2/version 8.0.0

Roxygen list(markdown = TRUE)

Contents

`adabay_accrual-methods` *Print method for objects of class* `adabay_accrual`

Description

Pretty-print the recruitment model (Poisson, piecewise, or callback), its rate (where applicable), the per-patient follow-up and the dropout specification.

Usage

```
## S3 method for class 'adabay_accrual'
print(x, ...)
```

Arguments

`x`	A `adabay_accrual` object from `set_accrual`.
`...`	Currently unused.

Value

The input, invisibly.

See Also

`set_accrual`.

adabay_cache-methods *Print method for objects of class* adabay_cache

Description

Pretty-print the endpoint, look count, virtual-trial count, and the sorted vector of effect-size thresholds cached for grid evaluation by evaluate_design or calibrate_design.

Usage

```
## S3 method for class 'adabay_cache'
print(x, ...)
```

Arguments

x	A adabay_cache object from build_cache.
...	Currently unused.

Value

The input, invisibly.

See Also

build_cache, evaluate_design, calibrate_design.

adabay_calibration-methods *Print method for objects of class* adabay_calibration

Description

Pretty-print the size of the evaluated threshold grid, followed by the selected design in $best rendered through print.adabay_calibration_best. When no grid point met the alpha_target and power_target constraints, $best is NULL and this is reported instead. The grid itself is not printed: it is a data frame of one row per threshold combination and is available as $grid.

Usage

```
## S3 method for class 'adabay_calibration'
print(x, ...)
```

Arguments

x	A adabay_calibration object from calibrate_design.
...	Currently unused.

Details

The object returned by calibrate_design is a list of two elements: $best, of class adabay_calibration_best, and $grid, a data frame with columns p, q, type_I, power and E_N. Printing the object without a method would emit the whole grid, which at a typical sweep is several hundred rows.

Value

The input, invisibly.

See Also

calibrate_design, build_cache, evaluate_design.

---

adabay_calibration_best-methods

Print method for objects of class adabay_calibration_best

Description

Pretty-print the selected threshold pair (p, q) together with the type I error rate under H_0, the power under H_1 and the expected sample size under H_1 at those thresholds. The futility threshold is reported as "disabled" when q >= 1. A note records that the reported type I error rate and power follow the calibration's futility convention: under the default non-binding rule the efficacy-crossing probability ignores futility crossings, so the full per-hypothesis operating characteristics in $oc_h0 and $oc_h1 should be consulted for the realised behaviour.

Usage

```
## S3 method for class 'adabay_calibration_best'
print(x, ...)
```

Arguments

x	A adabay_calibration_best object, the $best element returned by calibrate_design.
...	Currently unused.

Details

The object is the single grid point that meets the alpha_target and power_target constraints at minimum expected sample size under the alternative, selected from the user-supplied efficacy_grid by futility_grid sweep. It is deliberately not of class adabay_oc: its power field is the efficacy-crossing probability under H_1, whereas oc_h1$alpha carries the same quantity under the adabay_oc naming, and printing it through the adabay_oc method would mislabel it.

Value

The input, invisibly.

See Also

calibrate_design, build_cache, evaluate_design.

adabay_decision-methods *Print method for objects of class* adabay_decision

Description

Pretty-print the efficacy and futility rules, the binding-futility flag, and indicate whether the rules are common across looks or per-look.

Usage

```
## S3 method for class 'adabay_decision'
print(x, ...)
```

Arguments

x	A adabay_decision object from set_decision.
...	Currently unused.

Value

The input, invisibly.

See Also

set_decision.

adabay_design-methods *Print method for objects of class* adabay_design

Description

Pretty-print the endpoint, look schedule, effect scale, alternative, null-effect value and the per-arm cumulative sample-size (or exposure, or event-count) vectors of a Bayesian group sequential design skeleton.

Usage

```
## S3 method for class 'adabay_design'
print(x, ...)
```

Arguments

x	A adabay_design object from set_design.
...	Currently unused.

Value

The input, invisibly.

See Also

set_design.

---

adabay_oc-methods *Methods for objects of class* adabay_oc

Description

Print, summary, and per-look stopping-probability bar chart for an operating-characteristics object produced by evaluate_design. The plot uses ggplot2::theme_bw() and the BMJ palette via ggsci::scale_fill_bmj() when both packages are available, falling back to base graphics otherwise.

Usage

```
## S3 method for class 'adabay_oc'
print(x, ...)

## S3 method for class 'adabay_oc'
plot(x, file = NULL, width = 7, height = 4, dpi = 300, ...)

## S3 method for class 'adabay_oc'
summary(object, ...)
```

Arguments

x, object	A adabay_oc object returned by evaluate_design.
file	Optional output path. When supplied, the plot is written via ggplot2::ggsave; the file extension (.jpeg, .jpg, .png, .pdf) determines the device.
width, height	Plot dimensions in inches when file is supplied.
dpi	Resolution for raster outputs.
...	Additional arguments forwarded to ggsave (for plot) or ignored (for print and summary).

Value

print returns its input invisibly. summary returns a named list with elements alpha (overall type I error rate, under the regulatory convention for non-binding futility), eff_prob (overall efficacy probability under binding conduct), fut_prob (overall futility probability under binding conduct), ess, edur, n_trials, eff_prob_at_look (stage efficacy stopping probabilities) and fut_prob_at_look (stage futility stopping probabilities). plot returns a ggplot object (or NULL, invisibly, when ggplot2 is not installed).

See Also

evaluate_design, build_cache.

adabay_prior-methods *Print method for objects of class* adabay_prior

Description

Pretty-print the endpoint and the per-arm conjugate (possibly mixture) prior specification: family, component count, and per-component weights and hyperparameters.

Usage

```
## S3 method for class 'adabay_prior'
print(x, ...)
```

Arguments

x	A adabay_prior object from set_prior.
...	Currently unused.

Value

The input, invisibly.

See Also

set_prior, fit_mixture.

adabay_prior_arm-methods *Methods for objects of class* adabay_prior_arm

Description

print() reports the kernel family, the number of mixture components with each component's weight and conjugate parameters (in the same format as print.adabay_prior), the fit method, the empirical forward Kullback–Leibler divergence, the seed passed to fit_mixture (if any), and the per-quantile tail-probability error table. plot() overlays the empirical-sample density on the fitted mixture density and marks the tail-quantile thresholds as a rug.

Usage

```
## S3 method for class 'adabay_prior_arm'
print(x, ...)

## S3 method for class 'adabay_prior_arm'
plot(x, file = NULL, width = 7, height = 4, dpi = 300, ...)
```

Arguments

x	A adabay_prior_arm object returned by fit_mixture.
file	Optional output path. When supplied, the plot is written via ggplot2::ggsave; the file extension determines the device.
width, height	Plot dimensions in inches when file is supplied.
dpi	Resolution for raster outputs.
...	Additional arguments forwarded to ggsave (for plot) or ignored (for print).

Value

print returns its input invisibly. plot returns a ggplot object (or NULL, invisibly, when ggplot2 is not installed).

See Also

fit_mixture, set_prior.

---

build_cache *Build a precomputation cache for grid evaluation*

Description

Implements the precomputation strategy of Section 6.1 of the manuscript. Simulates n_trials virtual trials once at the union of all candidate look times, computes per-look posterior tail probabilities at all candidate effect-size thresholds, and stores the result. Subsequent calls to evaluate_design() or calibrate_design() then evaluate any candidate $(K, \boldsymbol{n}, p_{k,u}, q_{k,v})$ configuration without further simulation cost.

Usage

```
build_cache(
  design,
  prior,
  effect,
  accrual = NULL,
  n_trials = 10000L,
  cores = 1L,
  seed = NULL,
  threshold_grid = NULL,
  cross_core_reproducible = FALSE
)
```

Arguments

design	A adabay_design object whose schedule represents the union of all candidate look times.
prior	A adabay_prior object.
effect	Data-generating values, as in evaluate_design().

`accrual` Optional `set_accrual()` specification. **Required** for tte designs, which are always staggered: the recruitment rate fixes the Poisson enrolment process whose arrival times drive the calendar-time exposure cutoff. The tte endpoint currently supports only `model = "poisson"`. When supplied (any endpoint) it is stored in the cache so that `evaluate_design()` on the cache reports the expected duration (exact, from simulated calendar times, for tte; analytic for the other endpoints, which have no simulated recruitment timeline).

`n_trials` Number of virtual trials.

`cores` Number of CPU cores. Defaults to 1.

`seed` Optional integer seed.

`threshold_grid` Named list with elements `efficacy` and `futility`, each a numeric vector of effect-size thresholds to cache. Defaults to `list(efficacy = design$delta_null, futility = design$delta_null)`.

`cross_core_reproducible`
Logical, default `FALSE`. When `TRUE`, the simulator is forced to run in a single driver process (the user- supplied `cores` is ignored for the simulation pass) so that the per-trial RNG draws stored in the cache are bit-identical across worker counts. The flag belongs on the simulating call only: the cached path (`evaluate_design()` on an adabay_cache) and `calibrate_design()` re-use the stored draws and are already core-independent, so setting it there has no effect. See `evaluate_design()` for the cost discussion.

Value

An object of class `"adabay_cache"`.

Examples

```
des <- set_design(endpoint     = "binary",
                  n_per_look   = c(760, 1520, 2280),
                  effect_scale = "risk_difference",
                  alternative  = "less")
pri <- set_prior(endpoint = "binary",
                 arms = list(c = list(family = "beta", a = 1, b = 1),
                             t = list(family = "beta", a = 1, b = 1)))
cache <- build_cache(des, pri,
                     effect         = list(theta_c = 0.33, theta_t = 0.28),
                     n_trials       = 200L,
                     seed           = 1L,
                     threshold_grid = list(efficacy = 0, futility = 0))
print(cache)
```

`calibrate_design` *Calibrate design thresholds against operating-characteristic targets*

Description

Sweeps a grid of efficacy and futility posterior-probability thresholds against a precomputed cache, evaluating each candidate design and returning the design that minimises the expected sample size under the alternative subject to the type I error and power constraints.

Usage

```
calibrate_design(
  cache,
  cache_alt,
  alpha_target = 0.025,
  power_target = 0.9,
  efficacy_grid,
  futility_grid = c(1),
  futility_binding = FALSE
)
```

Arguments

`cache`	An object of class `"adabay_cache"` cached under H_0 (drives the type I error rate of every grid point).
`cache_alt`	A second `"adabay_cache"` cached under H_1 (drives the power and expected sample size of every grid point). Required: a calibration is a constrained search over both targets and cannot be carried out from a single cache.
`alpha_target`	Required upper bound on the type I error rate.
`power_target`	Required lower bound on the power (probability of crossing efficacy under the alternative effect).
`efficacy_grid`	Numeric vector of efficacy posterior-probability thresholds to search.
`futility_grid`	Numeric vector of futility posterior-probability thresholds to search. Use `c(1)` to disable futility.
`futility_binding`	Logical. Defaults to `FALSE`.

Details

This is a convenience wrapper around `evaluate_design()` that constructs the decision rule for each combination of efficacy threshold (`p`) and futility threshold (`q`) over the supplied grids, both applied at every look at the null effect-size value.

For more flexible calibration (per-look thresholds, dual-criterion rules, alternative effect-size cuts), call `evaluate_design()` directly within a user-defined search loop.

Value

An object of class `"adabay_calibration"`: a list with elements

`best` An object of class `"adabay_calibration_best"` (or `NULL`, with a warning, if no grid point meets both targets) for the `(p, q)` combination that meets the targets at minimum expected sample size under the alternative. Carries `p`, `q`, `type_I` (from the H_0 cache), `power` and `expected_sample_size` (both from the H_1 cache) as plain numeric fields, plus the full `adabay_oc` objects for each hypothesis as `oc_h0` / `oc_h1` for deeper inspection (e.g. `plot(cal$best$oc_h1)` for per-look stopping probabilities). Deliberately not of class `"adabay_oc"` itself: `oc_h1$alpha` is the *power* (it was evaluated under the alternative), and printing it through `print.adabay_oc()` would mislabel that as "Type I error".

`grid` A data frame of every grid point evaluated, with columns `p`, `q`, `type_I`, `power`, `E_N`.

Examples

```
des <- set_design(endpoint     = "binary",
                  n_per_look   = c(760, 1520, 2280),
                  effect_scale = "risk_difference",
                  alternative  = "less")
pri <- set_prior(endpoint = "binary",
                 arms = list(c = list(family = "beta", a = 1, b = 1),
                             t = list(family = "beta", a = 1, b = 1)))
cache_h0 <- build_cache(des, pri,
                        effect   = list(theta_c = 0.33, theta_t = 0.33),
                        n_trials = 200L, seed = 1L,
                        threshold_grid = list(efficacy = 0, futility = 0))
cache_h1 <- build_cache(des, pri,
                        effect   = list(theta_c = 0.33, theta_t = 0.28),
                        n_trials = 200L, seed = 1L,
                        threshold_grid = list(efficacy = 0, futility = 0))
cal <- calibrate_design(cache          = cache_h0,
                        cache_alt      = cache_h1,
                        alpha_target   = 0.05,
                        power_target   = 0.50,
                        efficacy_grid  = c(0.95, 0.99),
                        futility_grid  = c(0.80, 0.90))
head(cal$grid)
```

`evaluate_design` *Evaluate operating characteristics for a Bayesian group sequential design*

Description

`evaluate_design()` is the single entry point for obtaining the operating characteristics of a Bayesian group sequential design. It is polymorphic in its first argument:

Usage

```
evaluate_design(x, ...)

## Default S3 method:
evaluate_design(x, ...)

## S3 method for class 'adabay_design'
evaluate_design(
  x,
  prior,
  decision,
  effect,
  accrual = NULL,
  n_trials = 10000L,
  cores = 1L,
  seed = NULL,
  cross_core_reproducible = FALSE,
```

```
  ...
)

## S3 method for class 'adabay_cache'
evaluate_design(x, decision, look_subset = NULL, ...)
```

Arguments

`x`	A `adabay_design` object (fused path) or a `adabay_cache` object (cached path).
`...`	Further arguments passed to methods (currently unused).
`prior`	For the fused path only: a `adabay_prior` from `set_prior()`.
`decision`	A `adabay_decision` from `set_decision()`.
`effect`	For the fused path only: named list of per-arm data-generating values, named `mu_c`/`mu_t` for continuous, `theta_c`/`theta_t` (or `p_c`/`p_t`) for binary, and `lambda_c`/`lambda_t` for count and time-to-event. The generic aliases `psi_c`/`psi_t` are accepted for every endpoint.
`accrual`	For the fused path only: a `set_accrual()` specification. **Required** for tte designs, which are always staggered: the recruitment rate fixes the Poisson enrolment process whose arrival times drive the calendar-time exposure cutoff. The tte endpoint currently supports only `model = "poisson"`. Optional for the other endpoints (continuous, binary, count), which have no simulated recruitment timeline, where it is used only to report an analytic expected duration.
`n_trials`	For the fused path only: number of virtual trials.
`cores`	For the fused path only: number of CPU cores. Defaults to 1.
`seed`	For the fused path only: optional integer seed.
`cross_core_reproducible`	For the fused path only: logical, default `FALSE`. When `TRUE`, the simulator is forced to run in a single driver process (the user-supplied `cores` is ignored for the simulation pass) so that the trial-level RNG draws are bit-identical across worker counts. The posterior and aggregation passes are core-independent in both modes. Set to `TRUE` for regulatory submissions or other settings where reproducibility across hardware configurations is required; leave `FALSE` for design exploration where wall-clock matters more than cross-`cores` bit-identity. The cost is the loss of simulator parallelism: small for binary, count and tte (where the simulator is a small fraction of total wall-clock); moderate for continuous.
`look_subset`	For the cached path only: optional integer vector. If supplied, the design is evaluated at the subset of look indices given (in order). Defaults to all looks in the cached design.

Details

`adabay_design` Fused path: simulates `n_trials` virtual trials at the supplied data-generating values, computes per-look posterior tail probabilities of Δ (in closed form, or by fixed-node Gauss–Legendre quadrature on the binary and count rate-difference scales and by `stats::integrate()` on the continuous normal–inverse-gamma path), and applies the supplied decision rule to obtain the per-look stopping probabilities, the overall type I error rate (or power under the alternative), and the expected sample size, all in one call. Use this for one-off evaluation of a single design.

`adabay_cache` Cached path: looks up pre-computed per-look tail probabilities in a `adabay_cache` produced once by `build_cache()` and aggregates them under the supplied decision rule, without re-simulating trials. Use this for grid-style calibration when many decision rules share the same simulated trials.

Slow inner integrals are dispatched to C++ implementations via **Rcpp**, except on the continuous normal–inverse-gamma path, which uses `stats::integrate()` at the R level. The trial-level data-generating loop is embarrassingly parallel and is parallelised across `cores` workers via `parallel::mclapply()` (POSIX) or `parallel::parLapply()` (Windows); the posterior and aggregation passes are vectorised and run serially.

Value

An object of class `"adabay_oc"`, a list with elements:

`alpha` Overall type I error rate scalar (regulatory convention: under non-binding futility, computed as if futility were absent).

`eff_prob` Overall efficacy probability scalar under binding trial conduct.

`fut_prob` Overall futility probability scalar under binding trial conduct.

`eff_prob_at_look`, `fut_prob_at_look` Per-look stopping probabilities under binding conduct; they partition the sample space.

`alpha_at_look_binding`, `alpha_at_look_nonbinding` Alpha spent at each look under binding and non-binding semantics respectively.

`expected_sample_size` Expected total subjects (continuous, binary), exposure (count) or events (tte) under binding conduct.

`expected_duration` Expected calendar duration; present only when an `set_accrual()` specification is supplied (fused path) or was supplied to `build_cache()` (cached path), and absent otherwise. For tte this is the exact Monte Carlo mean of the calendar time realised (during simulation) at each trial's own stopping look; for continuous, binary and count – which have no simulated recruitment timeline – it is an analytic approximation.

`tau`, `C` Per-trial stopping look and trial outcome code: `+1` for an efficacy stop and `-1` for a futility stop or for reaching the final look without efficacy. The final analysis always forces a decision, so every trial ends at `+1` or `-1`.

`n_trials`, `effect`, `design`, `decision` Echoes of the inputs for downstream reproducibility. The fused path additionally records `seed` and `call`, which are omitted on the cached path, and `accrual`, which is carried whenever the cache was built with one.

Binding vs non-binding futility

The simulator always conducts virtual trials under binding semantics (a trial physically stops at the first crossing of either the efficacy or futility threshold). The per-look stopping probabilities `eff_prob_at_look[k]` and `fut_prob_at_look[k]` therefore always reflect actual binding conduct and partition the sample space: `sum(eff_prob_at_look) + sum(fut_prob_at_look) = 1` for both binding and non-binding settings of `futility_binding`.

The flag changes only the overall scalar type I error rate `alpha`:

Binding `alpha = sum(eff_prob_at_look)`: probability that the trial crosses efficacy before any futility crossing under the data-generating distribution.

Non-binding `alpha = sum(alpha_at_look_nonbinding)`: probability that the trial would have crossed efficacy at some look had the futility rule been ignored (the standard regulatory convention).

Examples

```
des <- set_design(endpoint = "binary",
                  n_per_look = c(760, 1520, 2280, 3040, 3800),
                  effect_scale = "risk_difference",
                  alternative = "less")
pri <- set_prior(endpoint = "binary",
                 arms = list(c = list(family = "beta", a = 1, b = 1),
                             t = list(family = "beta", a = 1, b = 1)))
dec <- set_decision(
  efficacy = list(list(threshold_effect = 0, threshold_prob = 0.99)),
  futility = list(list(threshold_effect = 0, threshold_prob = 0.90)),
  futility_binding = FALSE)

## Fused path: evaluate a single design directly
oc <- evaluate_design(des, prior = pri, decision = dec,
                      effect = list(theta_c = 0.33, theta_t = 0.28),
                      n_trials = 200L, cores = 1L, seed = 1L)
print(oc)

## Cached path: build once, evaluate many decision rules cheaply
cache <- build_cache(des, pri,
                     effect         = list(theta_c = 0.33, theta_t = 0.28),
                     n_trials       = 200L,
                     seed           = 1L,
                     threshold_grid = list(efficacy = 0, futility = 0))
oc2 <- evaluate_design(cache, dec)
print(oc2)
```

`fit_mixture` *Approximate a prior by a finite mixture of conjugate components*

Description

Approximates a user-supplied prior on an arm-specific parameter by a finite mixture of conjugate components (beta, normal or gamma kernels) fitted by minimisation of the empirical forward Kullback–Leibler divergence. The number of components is selected by `RBesT::automixfit()`'s AIC rule when the suggested **RBesT** package is installed. Otherwise the internal fallback locates an elbow in the divergence decrement – the smallest L past which adding a component no longer improves the divergence by at least `tol_kl` – and then continues past that elbow one component at a time until the maximum absolute tail-probability error falls within `tol_tail`, returning the smallest-error fit if none does. Either way the count is capped at `n_components_max`.

Usage

```
fit_mixture(
  endpoint,
  arm,
  prior,
  n_components_max = 5L,
  tol_kl = 0.001,
  tol_tail = 0.005,
  tail_quantiles = c(0.025, 0.05, 0.1, 0.5, 0.9, 0.95, 0.975),
```

```
  n_samples = 10000L,
  seed = NULL
)
```

Arguments

endpoint	One of "binary", "count", "tte" or "continuous". The endpoint determines the kernel family.
arm	Either "c" (control) or "t" (treatment), used as metadata on the returned object.
prior	Either a function prior(x) returning the density at x, or a numeric vector of independent samples drawn from the prior. A density function is accepted only for the beta kernel (binary endpoints), where n_samples draws are produced by inverse-transform sampling on a fine grid over $[0, 1]$. For the gamma and normal kernels the support is unbounded and no fixed grid is reliable, so prior must be a numeric vector of samples; supplying a function raises an error.
n_components_max	Maximum number of mixture components to try.
tol_kl	Mixture-fit tolerance, supplied on the natural (untransformed) scale, e.g. 1e-3. When **RBesT** is installed – the default path – it is forwarded to RBesT::automixfit() as the per-parameter EM convergence tolerance eps, which **RBesT** expresses on a $-\log_{10}$ scale, so tol_kl = 1e-3 is passed as eps = 3; the number of components is then chosen by the AIC rule inside RBesT::automixfit(). When **RBesT** is absent, the internal expectation-maximisation fallback uses tol_kl directly as the Kullback–Leibler decrement defining the elbow: the elbow is the smallest L past which adding a component no longer improves the divergence by at least tol_kl. That elbow is the starting point, not the answer, because the tol_tail stage may select a larger L.
tol_tail	Tolerance for the maximum absolute tail-probability error across the default tail thresholds; warning() fires when the error exceeds this value. Default 5e-3 (0.5 percentage points).
tail_quantiles	Numeric vector of quantile levels at which the tail-probability error is computed. Defaults to c(0.025, 0.05, 0.10, 0.50, 0.90, 0.95, 0.975).
n_samples	Number of prior samples to use when prior is given as a function. Defaults to 10^4.
seed	Optional integer seed. When prior is a density function, sample generation (stats::runif()-based inverse- transform sampling) draws from the ambient RNG stream by default, so the fitted mixture depends on whatever ran before this call; passing seed calls set.seed(seed) immediately beforehand so the result is reproducible regardless of prior RNG state or execution order. Has no effect when prior is already a numeric vector of samples (no randomness is drawn in that case). Default NULL leaves the RNG untouched (current behaviour).

Details

Forward KL is mass-covering: it pressures the approximation to put mass wherever the true prior puts mass, but it does not, on its own, guarantee accurate tail probabilities. Because Bayesian GSDs stop on posterior tail probabilities, fit_mixture() additionally reports the empirical tail-probability error of the fitted mixture at a set of quantile thresholds (defaults to the 0.025, 0.05, 0.10, 0.50, 0.90, 0.95 and 0.975 empirical quantiles), and emits a warning() when the maximum absolute tail-probability error exceeds tol_tail. The diagnostics are stored on the returned object and summarised by the print() and plot() methods.

If the suggested package **RBesT** is available, this function delegates to `RBesT::automixfit`, which provides robust kernel-specific EM updates for the beta, normal and gamma kernels. If **RBesT** is not available, a built-in EM fallback is used (currently for the beta kernel only); other kernels return an informative error.

Value

An object of class `"adabay_prior_arm"` suitable for `set_prior()`; carries `family`, `components`, `weights` and a `diagnostics` attribute that includes the empirical samples used, the fitted forward KL, the per-quantile tail-probability error table, the maximum absolute tail error, a density overlay grid, the `seed` used (if any), and a logical `warning_fired` flag.

Examples

```
## A logit-normal prior on the control rate, approximated by a beta
## mixture. 'seed' makes the fit reproducible without relying on
## ambient RNG state (prior is a density function here, so samples are
## drawn internally).
logit_normal_density <- function(theta) {
  z <- log(theta / (1 - theta))
  dnorm(z, mean = -0.7, sd = 0.4) / (theta * (1 - theta))
}
pri_c <- fit_mixture(endpoint = "binary", arm = "c",
                     prior = logit_normal_density,
                     n_components_max = 5, tol_kl = 1e-3, seed = 1L)
print(pri_c)
plot(pri_c)
set_prior(endpoint = "binary",
          arms = list(c = pri_c,
                      t = list(family = "beta", a = 1, b = 1)))
```

`set_accrual` *Set accrual: recruitment, follow-up and dropout*

Description

Configures the recruitment process and the per-patient follow-up duration used to compute the expected study duration alongside the operating characteristics. The follow-up term enters the reported duration for the sample-size-driven continuous and binary schedules only; the exposure-driven count schedule and the event-driven time-to-event schedule do not add it. `dropout` and `dropout_rate` are accepted and stored for forward compatibility but are not consumed by any simulator or reported quantity in this release.

Usage

```
set_accrual(
  model = c("poisson", "piecewise", "callback"),
  rate = NULL,
  breakpoints = NULL,
  callback = NULL,
  follow_up = 0,
  dropout = c("none", "exponential"),
```

```
  dropout_rate = NULL
)
```

Arguments

`model`	Recruitment model. One of: `"poisson"` homogeneous Poisson process with constant rate `rate`. `"piecewise"` piecewise-constant Poisson process with breakpoints in `breakpoints` and rates in `rate`. `"callback"` user-supplied callback that returns recruitment times for a given target sample size. Only `"poisson"` affects the reported expected duration: the other models are accepted for specification, but `expected_duration` is returned as `NA` for them, and time-to-event designs require `"poisson"` outright.
`rate`	Recruitment rate, pooled across both arms (matching `n_per_look`/`exposure_per_look`/`d_per_look` in `set_design`, which are likewise pooled). Split into per-arm rates by the design's `allocation_ratio`: `rate_c = rate / (1 + allocation_ratio)`, `rate_t = rate_c * allocation_ratio`, so `rate_c + rate_t == rate` exactly. With the default `allocation_ratio = 1` this is a 1:1 split.
`breakpoints`	Numeric vector of calendar-time breakpoints (only used with `model = "piecewise"`).
`callback`	Function with signature `function(n, ...)` returning recruitment times (only used with `model = "callback"`).
`follow_up`	Per-patient follow-up duration: a single non-negative numeric. A function-valued `follow_up` is not supported in this release and is rejected with an informative error.
`dropout`	Dropout model. One of `"none"` (the default, administrative censoring at the look times) or `"exponential"` with rate `dropout_rate`.
`dropout_rate`	Per-arm dropout rate when `dropout = "exponential"`. Length-1 (pooled) or length-2 numeric.

Value

An object of class `"adabay_accrual"`.

Examples

```
set_accrual(model = "poisson", rate = 40)
```

---

set_decision *Set per-look decision rules*

Description

Defines the efficacy and futility stopping rules for a Bayesian group sequential design. A decision rule is a list of one or more criteria, each a list with elements `threshold_effect` (effect-size threshold $e_{k,u}$ or $f_{k,v}$) and `threshold_prob` (posterior-probability threshold $p_{k,u}$ or $q_{k,v}$). All criteria must be simultaneously met for the rule to fire.

Usage

```
set_decision(efficacy, futility = NULL, futility_binding = FALSE)
```

Arguments

efficacy	List of efficacy criteria (each with elements threshold_effect and threshold_prob), or a list of such lists, one per look.
futility	List of futility criteria, with the same structure as efficacy. Pass NULL or an empty list to disable futility stopping.
futility_binding	Logical. The simulator always conducts the trial under binding behaviour (trials stop on either efficacy or futility). This flag changes only the reported type I error rate. When TRUE, the type I error rate counts only efficacy crossings that occur before any futility crossing under the data-generating distribution. When FALSE (the default), the type I error rate is reported as the rate that would be observed if the futility rule had been ignored, which is identical to a design with no futility rule at all. The expected sample size, the futility probability, the per-look stopping probabilities and the trial-level outcomes tau and C are unchanged by the flag.

Details

Rules can vary per look by passing a list of lists with one element per look (length K, the number of looks). Supplying a single list applies the same rule at every look.

Value

An object of class "adabay_decision".

Examples

```
set_decision(
  efficacy = list(list(threshold_effect = 0, threshold_prob = 0.99)),
  futility = list(list(threshold_effect = 0, threshold_prob = 0.90)),
  futility_binding = FALSE)
```

---

set_design *Set the Bayesian group sequential design skeleton*

Description

Declares the endpoint type, the look schedule, and the pooled (both-arms) sample size, exposure or event-count target at each look, plus the effect scale and the direction of the alternative hypothesis.

Usage

```
set_design(
  endpoint,
  n_per_look = NULL,
  exposure_per_look = NULL,
  d_total = NULL,
```

```
  d_per_look = NULL,
  effect_scale = NULL,
  alternative = c("greater", "less"),
  delta_null = NULL,
  sigma = NULL,
  allocation_ratio = 1
)
```

Arguments

endpoint	One of "continuous", "binary", "count" or "tte" (time-to-event).
n_per_look	Cumulative sample size at each look, pooled across both arms (continuous and binary endpoints). The number of looks K is taken from length(n_per_look) – there is no separate n_looks argument. Split into per-arm targets by allocation_ratio: n_c[k] = round(n_per_look[k] / (1 + allocation_ratio)), n_t[k] = n_per_look[k] - n_c[k], so n_c[k] + n_t[k] == n_per_look[k] exactly at every look. With the default allocation_ratio = 1 this is a 1:1 split (up to rounding). Errors if consecutive n_per_look values are too close together for the split to be strictly increasing in both arms at every look.
exposure_per_look	Cumulative exposure at each look, pooled across both arms (count endpoint); K is taken from length(exposure_per_look). Split into per-arm targets by allocation_ratio the same way as n_per_look (exactly, with no rounding, since exposure is not required to be an integer).
d_total	Total target number of events for the tte endpoint, pooled across both arms; unaffected by allocation_ratio. Defaults to NULL, in which case the final entry of d_per_look is used.
d_per_look	Cumulative number of events at each look (tte endpoint), pooled across both arms; K is taken from length(d_per_look). Unaffected by allocation_ratio.
effect_scale	Effect-size parameterisation. One of "mean_difference" or "standardised_mean_difference" (continuous); "risk_difference", "risk_ratio" or "odds_ratio" (binary); "rate_difference", "rate_ratio" or "log_rate_ratio" (count); or "hazard_ratio" or "log_hazard_ratio" (time-to-event).
alternative	Direction of the alternative hypothesis, "greater" (default) or "less".
delta_null	Null-hypothesis effect-size value (Δ_{H_0} in the manuscript). Defaults to the no-effect value of the chosen effect_scale: 0 for the difference scales ("mean_difference", "standardised_mean_difference", "risk_difference", "rate_difference") and for the log scales ("log_rate_ratio", "log_hazard_ratio"), and 1 for the ratio scales ("risk_ratio", "odds_ratio", "rate_ratio", "hazard_ratio").
sigma	Common within-arm standard deviation of the simulated outcome data. Required for every continuous design, since it drives the Monte Carlo data generation, and additionally used by the known-variance normal prior.
allocation_ratio	Treatment-to-control allocation ratio, with a uniform meaning across all four endpoints: continuous/binary split n_per_look into per-arm sample-size targets by this ratio; count splits exposure_per_look the same way; tte scales the treatment arm's Poisson recruitment rate (see set_accrual) by this ratio relative to the control arm's, so the pooled d_per_look/d_total targets are reached under an unbalanced arm composition rather than a 1:1 split.

Value

An object of class "adabay_design".

Examples

```
set_design(endpoint = "binary",
           n_per_look = c(760, 1520, 2280, 3040, 3800),
           effect_scale = "risk_difference",
           alternative = "less")
```

`set_prior` *Set per-arm priors for a Bayesian group sequential design*

Description

Constructs a prior specification matched to the endpoint of the design. Each arm is given a (possibly mixture) conjugate prior. Pure conjugate priors are passed in directly; non-conjugate priors are first approximated by a finite mixture of conjugate components via `fit_mixture()` and then passed in as the resulting object.

Usage

```
set_prior(endpoint, arms)
```

Arguments

`endpoint`	One of `"continuous"`, `"binary"`, `"count"` or `"tte"` (time-to-event).
`arms`	A list with named elements `c` (control) and `t` (treatment), each a per-arm prior specification (see Details).

Details

Supported conjugate kernels per endpoint:

continuous `family = "normal"` with `mean` and `sd` (known variance), or `family = "normal_inverse_gamma"` with `nu, kappa, alpha, beta` (unknown variance).

binary `family = "beta"` with `a` and `b`.

count, tte `family = "gamma"` with `a` (shape) and `b` (rate).

Each per-arm specification can also be a multi-component mixture supplied as a list with element `components` (list of conjugate components) and `weights` (numeric vector summing to 1).

On the effect scales evaluated by fixed-node Gauss–Legendre quadrature (`"risk_difference"`, `"risk_ratio"` and `"odds_ratio"` for binary endpoints, and `"rate_difference"` for count endpoints), the posterior tail probability loses accuracy when a posterior shape parameter falls below 1, as can happen at early looks under a prior shape below 1 (for example a Jeffreys prior); `evaluate_design()` and `build_cache()` warn in that case. Prefer a prior whose shape parameters are all at least 1, or, for count endpoints, a relative-effect scale (`"rate_ratio"` or `"log_rate_ratio"`), which uses an exact closed form at any shape.

Value

An object of class "adabay_prior".

Examples

```
set_prior(endpoint = "binary",
          arms = list(c = list(family = "beta", a = 1, b = 1),
                      t = list(family = "beta", a = 1, b = 1)))
```

---

summarise_oc *Summarise operating characteristics in tidy form*

Description

Returns a single-row data.frame summarising the operating characteristics of one adabay_oc object, or a multi-row data.frame when supplied with a list of adabay_oc objects.

Usage

```
summarise_oc(x)
```

Arguments

x	A adabay_oc object or a list of them.

Details

alpha is the overall type I error rate (regulatory convention: under non-binding futility, computed as if futility were absent). eff_prob is the overall probability of stopping for efficacy under actual (binding) trial conduct – the power when the simulation was run under the alternative effect, and a coincident value with alpha under binding futility. fut_prob is the overall probability of stopping for futility under actual binding conduct.

Value

A data.frame with columns alpha (overall type I error), eff_prob (overall efficacy probability), fut_prob (overall futility probability), E_N (expected sample size / exposure / events under binding trial conduct), E_T (expected calendar duration when an set_accrual specification is supplied, otherwise NA), n_trials, n_looks, and the list-columns eff_prob_at_look and fut_prob_at_look carrying the stage stopping probability vectors.

Examples

```
des <- set_design(endpoint     = "binary",
                  n_per_look   = c(760, 1520, 2280),
                  effect_scale = "risk_difference",
                  alternative  = "less")
pri <- set_prior(endpoint = "binary",
                 arms = list(c = list(family = "beta", a = 1, b = 1),
                             t = list(family = "beta", a = 1, b = 1)))
dec <- set_decision(
  efficacy = list(list(threshold_effect = 0, threshold_prob = 0.99)),
  futility = list(list(threshold_effect = 0, threshold_prob = 0.90)),
```

```
  futility_binding = TRUE)
oc <- evaluate_design(des, pri, dec,
                      effect   = list(theta_c = 0.33, theta_t = 0.28),
                      n_trials = 200L, seed = 1L)
summarise_oc(oc)
```

Index